

Leitura Detalhada por ID via API

Evoluindo a leitura de dados: como buscar um registro específico no backend usando rotas parametrizadas, validação de existência e respostas HTTP adequadas.

CRUD

REST API

NESTJS

Missão: Encontrando um Registro Específico

Onde estamos

Já implementamos a listagem de todos os registros.

Agora, precisamos ir além: buscar **um único registro** a partir do seu identificador.

Por que isso importa?

A leitura por ID é uma operação essencial em APIs REST. Ela permite:

- Consultar os detalhes de um recurso específico
- Verificar se um registro realmente existe
- Confirmar alterações feitas por endpoints de atualização
- Preparar o sistema para exclusão segura e validações completas

Justificativa Pedagógica

Este material ajuda o estudante a compreender que a leitura de uma coleção de registros **não resolve todos os cenários** de uma API. Em sistemas corporativos, muitas operações dependem da identificação precisa de uma entidade.



Produto pelo ID

Buscar um produto específico para exibir seus detalhes completos



Pedido específico

Verificar os dados de um pedido antes de processá-lo



Categoria antes de alterar

Consultar uma categoria antes de modificá-la com segurança



Confirmar antes de remover

Verificar se um item existe antes de executar a exclusão

Objetivos de Aprendizagem

1

Criar o endpoint

Implementar um endpoint de leitura detalhada por ID na API

2

Receber parâmetros

Receber parâmetros de rota no controller com `@Param`

3

Buscar no service

Buscar um registro específico no service com validação

4

Retornar erro 404

Retornar erro adequado quando o item não existir no banco

5

Testar com REST Client

Testar a operação usando REST Client com cenários reais

6

Apoiar o CRUD

Utilizar a consulta como apoio para atualização, exclusão e validação

Listar Todos vs. Buscar por ID

Entender a diferença entre os dois tipos de leitura é fundamental para construir uma API bem estruturada.

GET /products — Listar todos

Retorna uma **coleção** de registros. Responde à pergunta: *"Quais registros existem no sistema?"*

```
[
  {
    "id": "1",
    "name": "Produto A"
  },
  {
    "id": "2",
    "name": "Produto B"
  }
]
```

Útil para visualizar o conjunto completo de dados disponíveis.

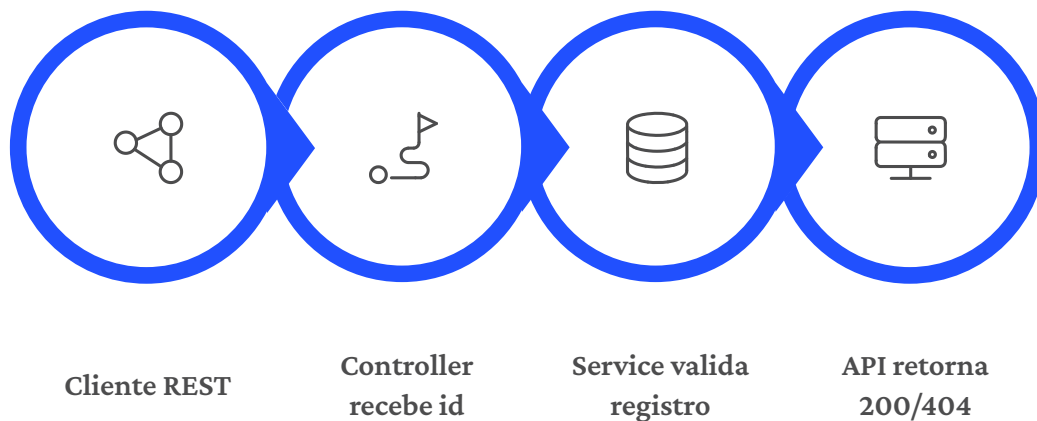
GET /products/:id — Buscar por ID

Retorna **apenas um registro**. Responde à pergunta: *"Quais são os dados deste registro específico?"*

```
{  
  "id": "1",  
  "name": "Produto A",  
  "description": "Descrição detalhada do produto A",  
  "price": 10.5  
}
```

Útil para trabalhar com um item específico com todos os seus detalhes.

Fluxo Completo da Leitura por ID



O fluxo se divide em dois caminhos: quando o registro **existe**, a API retorna o objeto com status 200. Quando o registro **não existe**, o service lança uma exceção e a API responde com 404 Not Found – informando claramente que o recurso solicitado não foi encontrado.

✓ Registro encontrado

- REST Client
- GET /products/:id
- Controller recebe o id
- Service busca o registro
- Service encontra o registro
- API retorna 200 OK

✗ Registro não encontrado

- REST Client
- GET /products/:id
- Controller recebe o id
- Service busca o registro
- Service NÃO encontra
- API retorna 404 Not Found

Implementação: Controller e Service

A implementação segue o princípio de separação de responsabilidades: o controller recebe o parâmetro e delega; o service concentra a lógica de busca e validação.

Controller — Recebendo o parâmetro

O controller usa o decorator `@Param` para extrair o `id` da rota e encaminha ao service:

```
@Get('/:id')
findOne(@Param('id') id: string) {
  return this.productsService.findOne(id);
}
```

O controller não acessa o banco diretamente. Ele apenas recebe e delega.

Service — Buscando e validando

O service concentra a lógica: busca o registro e lança erro se não encontrar:

```
async findOne(id: string) {
  const product = await this.productsRepository.findOne({where: { id },});

  if (!product) {
    throw new NotFoundException('Produto não encontrado.');
```

Por que Retornar 404?

Status HTTP 404 Not Found

Quando a API recebe um `id` válido do ponto de vista da rota, mas não encontra nenhum registro correspondente no banco de dados, o status mais adequado é **404 Not Found**.

Esse status informa claramente que o recurso solicitado não foi encontrado.

Exemplo de resposta esperada

```
GET /products/00000000-0000-0000-0000-000000000000
```

```
{
  "statusCode": 404,
  "message": "Produto não encontrado.",
  "error": "Not Found"
}
```

Retornar `null` ou uma resposta vazia sem contexto pode dificultar o diagnóstico por quem consome a API. Sempre forneça uma mensagem clara.

Testes com REST Client

Os testes devem cobrir os principais cenários da operação. Não teste os endpoints de forma isolada — encadeie as operações para simular um caso de uso real.

Teste 1 — Registro existente

Copie o `id` de um item existente, defina a variável e execute:

```
@productId = cole_um_id_valido

### [READ BY ID 1]
GET {{baseUrl}}/products/{{productId}}
```

Verifique: status 200, corpo com um único registro, `id` correto na resposta.

Teste 2 — Registro inexistente

Teste com um identificador que não existe no banco:

```
### [READ BY ID 2]
GET {{baseUrl}}/products/00000000-0000-0000-0000-000000000000
```

Verifique: status 404, mensagem de erro compreensível, sem retorno de `null`.

Teste 3 — Confirmar atualização

Use o GET por ID antes e depois de um PATCH para confirmar persistência:

```
### Consulta ANTES
GET {{baseUrl}}/products/{{productId}}

### Atualiza
PATCH {{baseUrl}}/products/{{productId}}

### Consulta DEPOIS
GET {{baseUrl}}/products/{{productId}}
```

Cuidados, Checklist e Próximos Passos

⚠ Ordem das rotas

Em alguns frameworks, a ordem de declaração das rotas pode causar conflito. Declare rotas mais específicas **antes** de rotas parametrizadas:

```
@Get('active')
findActive() {
  return this.productsService.findActive();
}

@Get(':id')
findOne(@Param('id') id: string) {
  return this.productsService.findOne(id);
}
```

✅ Checklist de implementação

Endpoint GET /products/:id criado no controller

Decorator @Param('id') utilizado corretamente

Método findOne(id) implementado no service

Validação com NotFoundException quando registro não encontrado

Testes com REST Client cobrindo cenário 200 e 404

Rota parametrizada declarada após rotas fixas